

Checking the strings

```
df1['Description'] =
df1['Description'].str.replace('.', '').
str.upper().str.strip()
df1['Description'] =
df1['Description'].replace('\s+', ' ',
regex = True)
df1['InvoiceNo'] = df1['InvoiceNo'].
astype(str).str.upper()
df1['StockCode'] = df1['StockCode'].
str.upper()
df1['Country'] = df1['Country'].str.
upper()
df1.head()
```

As intuitive as the variables (column names) may sound, let's take a step further by understanding what each variable means.

InvoiceNo (invoice_num): A number assigned to each transaction

StockCode (stock_code): Product code

Description (description): Product name

Quantity (quantity): Number of products purchased for each transaction

InvoiceDate (invoice_date): Time stamp for each transaction

UnitPrice (unit_price): Product price per unit

CustomerID (cust_id): Unique identifier for each customer

Country (country): Country name



Note: Product price per unit is assumed to follow the same currency throughout our analysis.

By understanding the data in a more descriptive manner, we notice two things:

- Quantity has negative values
- Unit price has zero values (Are these FREE items?)

We have some odd and irregular values in the 'UnitPrice' and 'Quantity' columns, as seen in the summary of our data set, which we will locate and eliminate to prevent them from negatively affecting our study. We can

see that some of the transactions in the 'StockCode' variable are not actually products, but rather costs or fees related to the post or bank, or other transactions that we don't really require in our data. We realise that some of these transactions contain returned products, and that in those transactions, the 'InvoiceNo' begins with a 'c' character and the 'UnitPrice' should be negative. However, we have purchases in our database with negative 'UnitPrice' and vice versa, which we need to correct. There are also certain tuples where the 'UnitPrice' is left blank. There are many missing or inaccurate values in the 'Description' attribute. To resolve this issue, we will delete transactions with no accessible description, check the 'Description' based on the product 'StockCode', and fill the missing values with the right 'Description' that is available from other transactions with the same 'StockCode'.

Next, we will look for the missing and incorrect values:

```
df1.drop(df1[(df1.Quantity>0) & (df1.
InvoiceNo.str.contains('C') == True)].
index, inplace = True)
df1.drop(df1[(df1.Quantity<0) & (df1.
InvoiceNo.str.contains('C') == False)].
index, inplace = True)
df1.drop(df1[df1.Description.str.
contains('?', regex=False) == True].
index, inplace = True)
df1.drop(df1[df1.UnitPrice == 0].index,
inplace = True)
```

```
for index,value in df1.StockCode[df1.
Description.isna()==True].items():
    if pd.notna(df1.Description[df1.
StockCode == value]).sum() != 0:
        df1.Description[index] = df1.
Description[df1.StockCode == value].
mode()[0]
    else:
        df1.drop(index = index, inplace
= True)
```

```
df1['Description'] =
df1['Description'].astype(str)
```

#Adding Desired Features

```
df1['FinalPrice'] = df1['Quantity']*df1
['UnitPrice']
df1['InvoiceMonth'] =
df1['InvoiceDate'].apply(lambda x:
x.strftime('%B'))
df1['Day of week'] =
df1['InvoiceDate'].dt.day_name()
df1.shape
```

The output of the execution will be:

(406789, 11)

Exploratory data analysis

In this section, we will visualise the data to have a clear vision and gain insights into it.

We import our data in this section and update to datetime so that we can work on the data from the time series. We're going to start by showing what the top items we have sold around the globe are. There are two metrics that can show us how much benefit each product has produced. In the first plot of the subplot below, we can see the top 20 goods purchased by clients with respect to price and in the most quantities.

```
#importing necessary libraries and the
cleaned dataset
import pandas as pd, numpy as np,
matplotlib.pyplot as plt, seaborn as
sns
%matplotlib inline
Cleaned_Data = df1
Cleaned_Data.index = pd.to_
datetime(Cleaned_Data.index, format =
'%Y-%m-%d %H:%M')
```

```
#top 20 products by quantity and
finalprice
sns.set_style('whitegrid')
TopTwenty = Cleaned_Data.
groupby('Description')['Quantity'].
agg('sum').sort_values(ascending=False)
[0:20]
```

```
Top20Price = Cleaned_Data.
groupby('Description')['FinalPrice'].
```